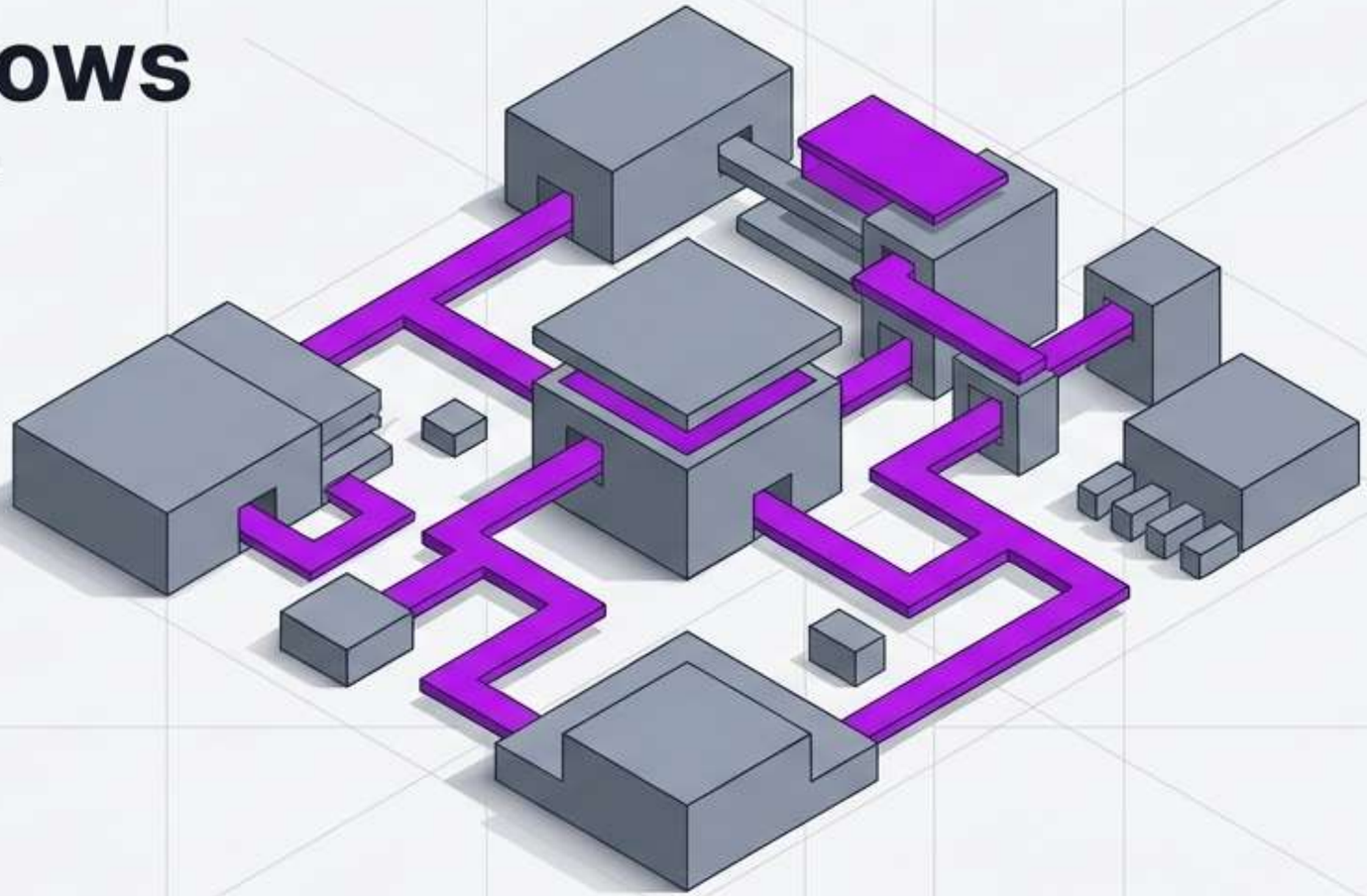
Building Real App Flows with the App Router

CCA Full-Stack Development
Session 02: Loading States, Route Handlers, Forms, Data Flow



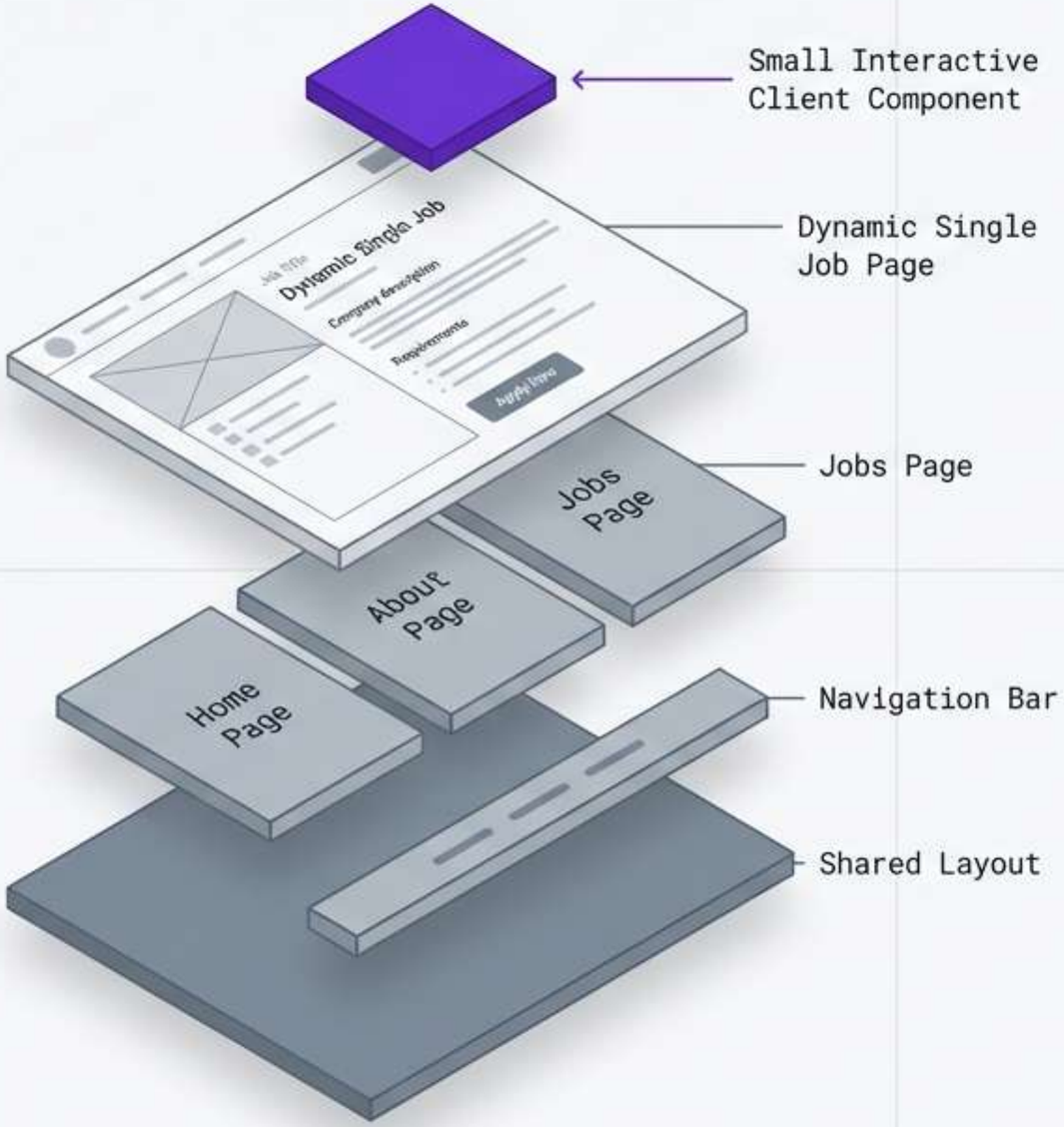
CODEZELA CAREER ACCELERATOR
FULL-STACK DEVELOPER PATHWAY
NEXT.JS CLASS — SESSION 02
INSTRUCTOR: SAYURU AMARASINGHE

NotebookLM

The Static Foundation Built in Session 01

In our previous session, we constructed the structural layers of our application. We established routing, shared layouts, and basic mock data.

We already built the foundation. Today we make it behave like a real product.



NotebookLM

The Mindset Shift: From Page Builder to Product Developer

Session 01: Page Builder	Session 02: Product Developer
Pages & Routes	**User Journeys**
Single Files	**Separated Logic**
Raw Mock Arrays	**Data Helpers**
Static Links & Clicks	**Form Submissions**
Presenting Information	**Executing Actions**

Last time, we focused on **structure**.
Today, we focus on behavior.

NotebookLM

The Connected User Journey for DevJobs

View: User opens Jobs page & selects a job.

Wait: App loads full job details.

Apply: User fills out the application form.

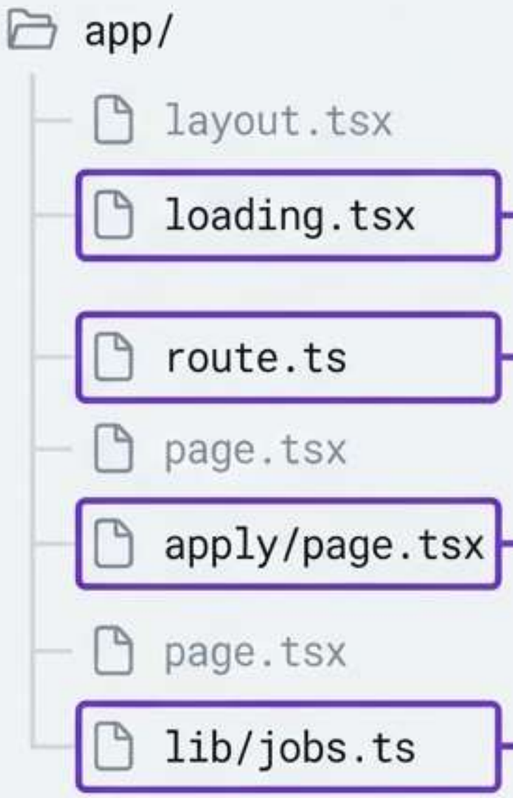
Submit: System processes the application and responds.

Good route structure naturally supports the user journey. Instead of building disconnected pages, we are wiring together a continuous flow.

NotebookLM

The Architecture of a Growing Application

A better structure now makes future features easier later.
Pages should focus purely on UI, while data and logic are extracted out.



`loading.tsx` | Intercepts wait times to show loading UI for a route segment.

`route.ts` | Creates backend-style route handlers directly inside Next.js.

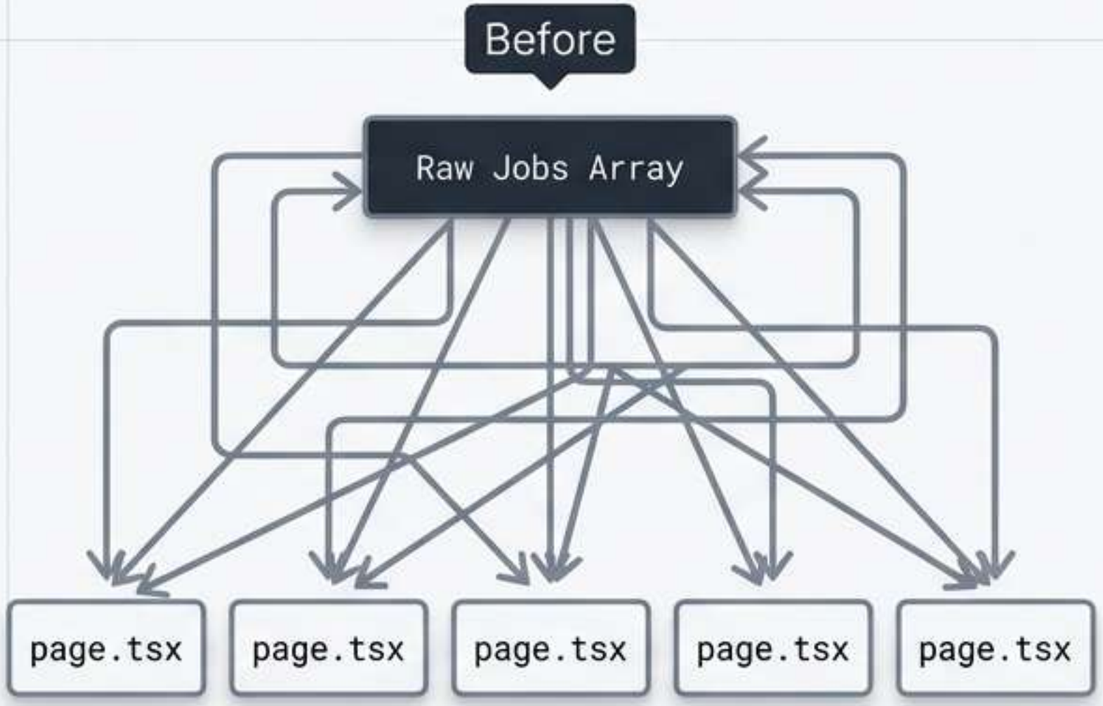
`apply/page.tsx` | Renders the interactive job application page.

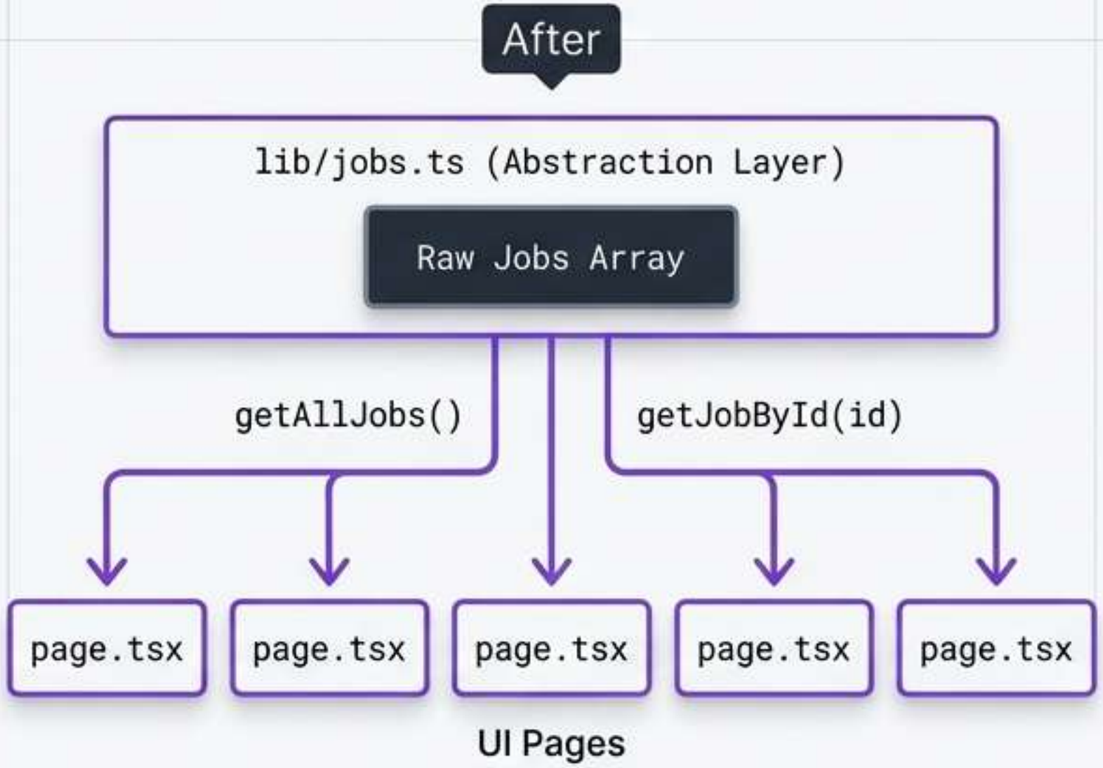
`lib/jobs.ts` | Moves raw data access into a cleaner, dedicated helper layer.

NotebookLM

Abstracting Data for Scalability

Do not import raw data everywhere. Directly utilizing arrays in every page creates brittle code.





Benefits:

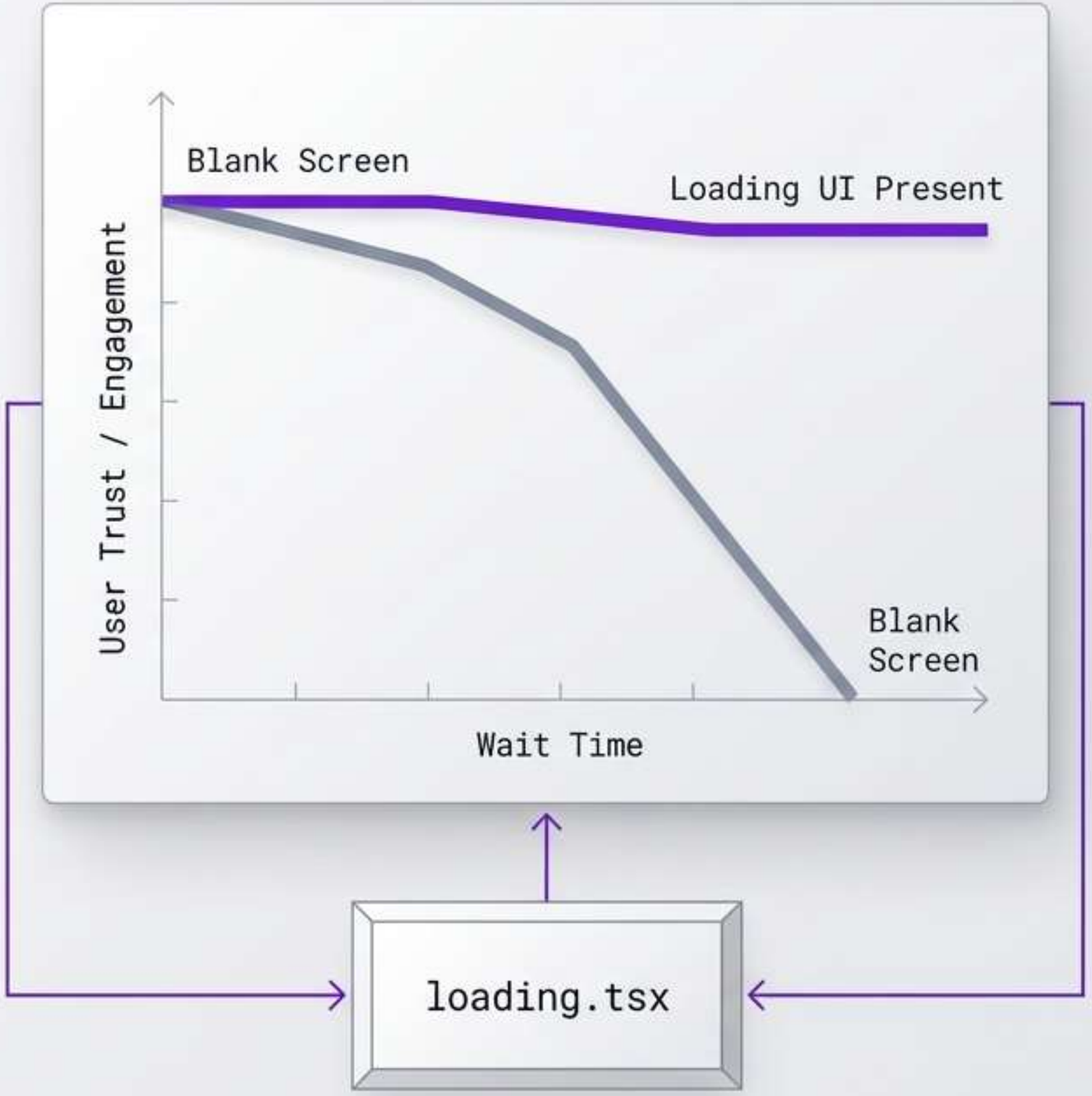
- Cleaner code, better separation of concerns, easier future API integration.

NotebookLM

The Psychology of Loading States

Real apps should never feel ‘dead’ while computing. A blank screen creates user doubt; immediate feedback creates trust.

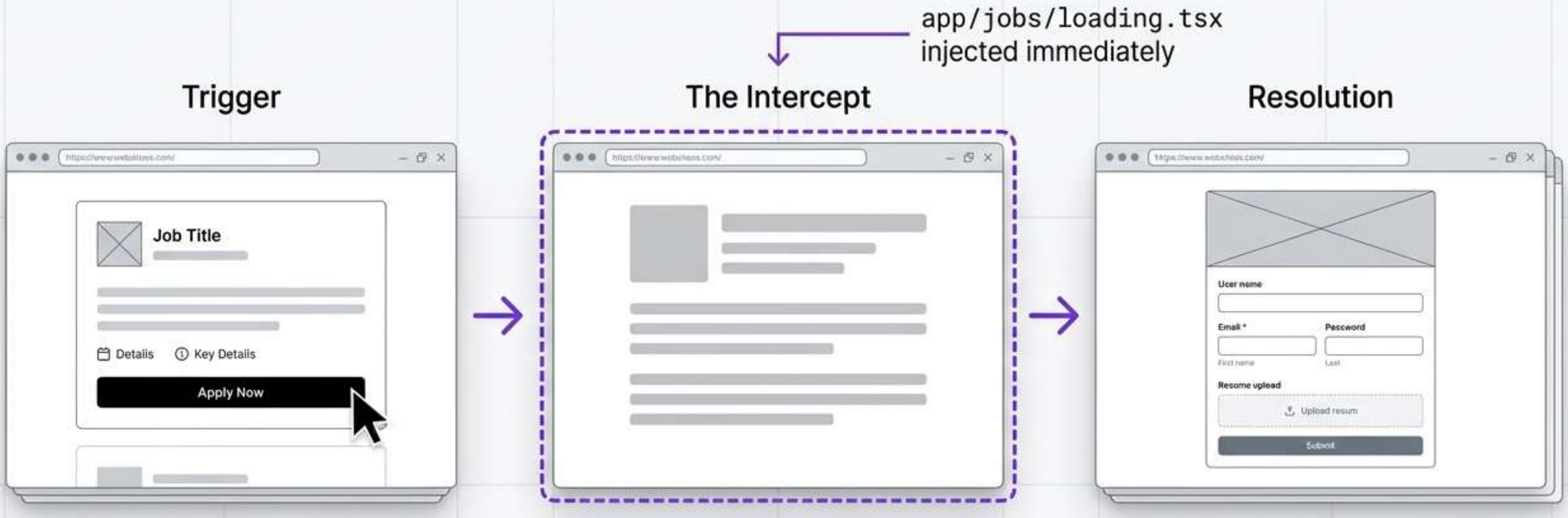
Key Concept: The Next.js App Router automatically supports `route-level loading` through a dedicated `loading.tsx` file.



NotebookLM

How Route-Level Loading Works in Practice

By instantly showing a structural placeholder, we improve the perceived performance of the application, keeping the user engaged while the server fetches heavy data.

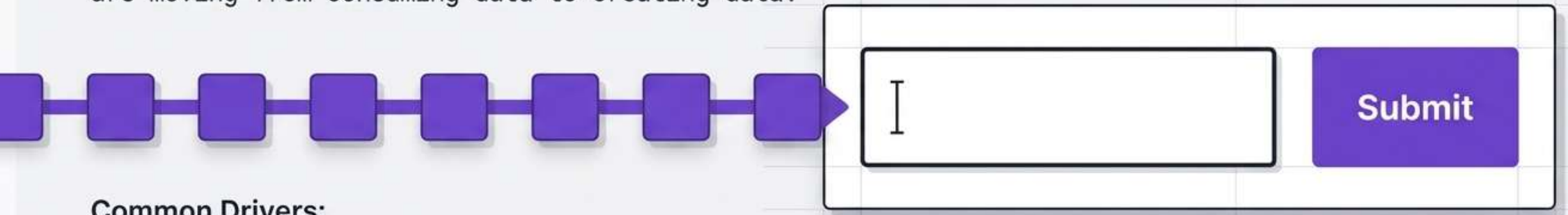


By instantly showing a structural placeholder, we improve the perceived performance of the application, keeping the user engaged while the server fetches heavy data.

NotebookLM

Forms Are the Engine of Application Interactivity

Most real systems only become useful when users can input data and trigger actions. We are moving from consuming data to creating data.



Common Drivers:
Contact forms, logins, profile updates, bookings.

Today's Focus: Building the interactive "Apply" flow for the DevJobs platform.

NotebookLM

Diagnosing the UI: When to Use Client Components

Rule of Thumb: Use client-side logic only where interaction is strictly needed. Keep a server-first mindset.

Does this UI element require real-time interaction?

No

Server Component (Default)

- Fetching job data
- Rendering structural layouts
- Optimizing SEO needs

Yes

Client Component (Exception)

- Tracking form inputs
- Handling onSubmit events
- Triggering loading buttons
- Rendering success/error messages

NotebookLM

Route Handlers: Your Integrated Backend

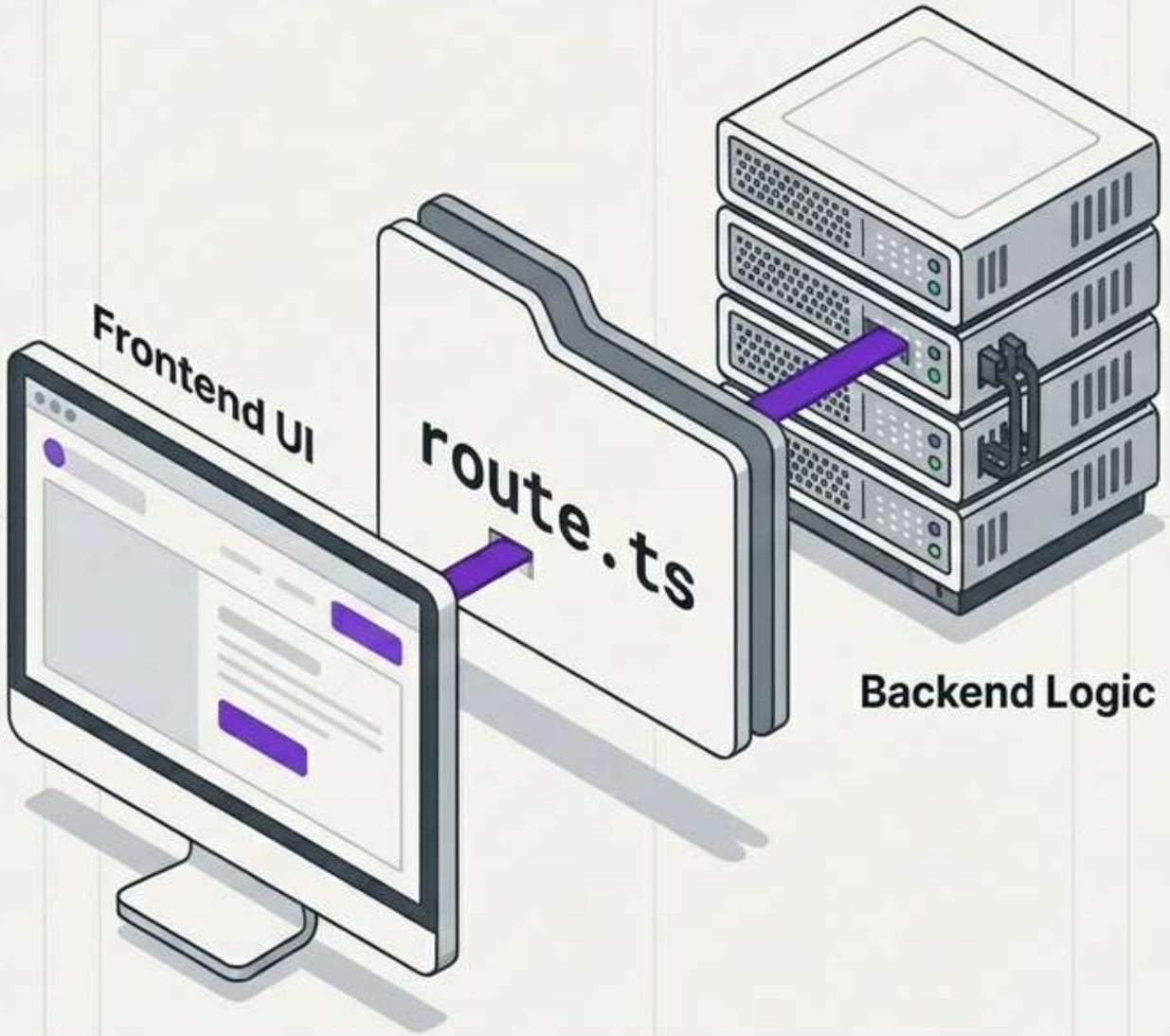
Next.js route handlers (route.ts) allow us to create robust, backend-style API endpoints within the exact same project structure as our UI.

Core Capabilities:

- Receive incoming HTTP requests
- Process complex data safely on the server
- Return standardized JSON responses

Use Cases:

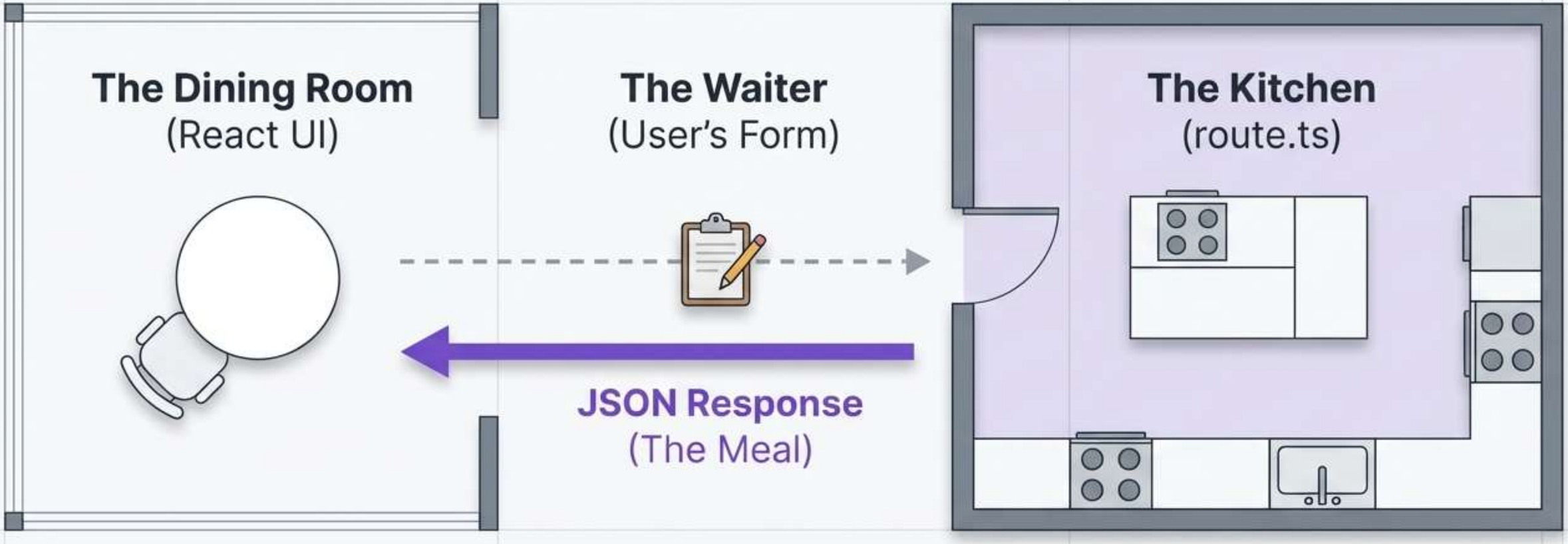
Getting job data, handling internal logic, and securely receiving form submissions.



NotebookLM

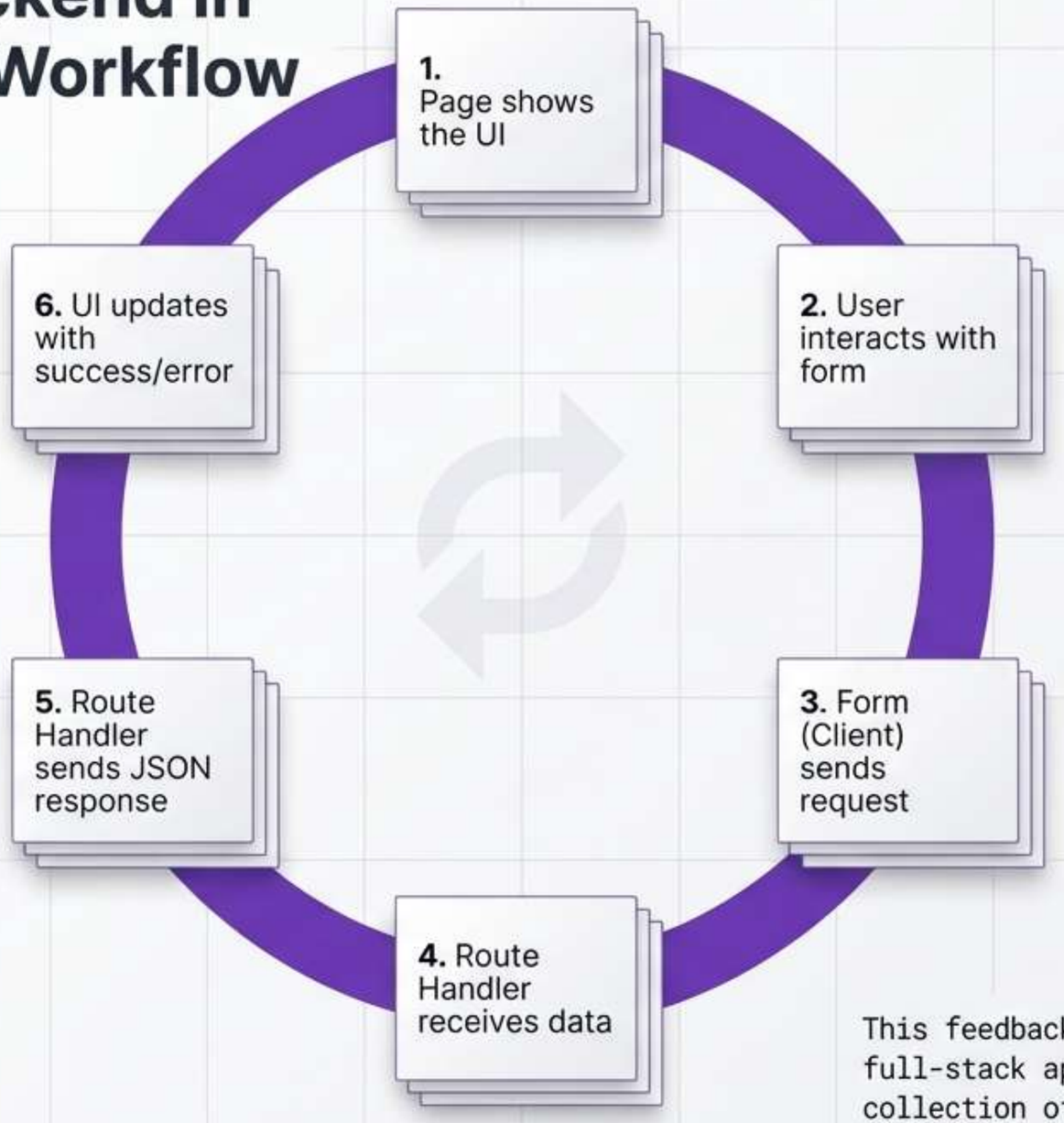
Demystifying the Client-Server Boundary

The React UI is the dining room where the user sits. The form is the waiter taking their order. The route.ts file is the kitchen—securely processing the request out of sight and returning the final result.



NotebookLM

Frontend and Backend in One Continuous Workflow

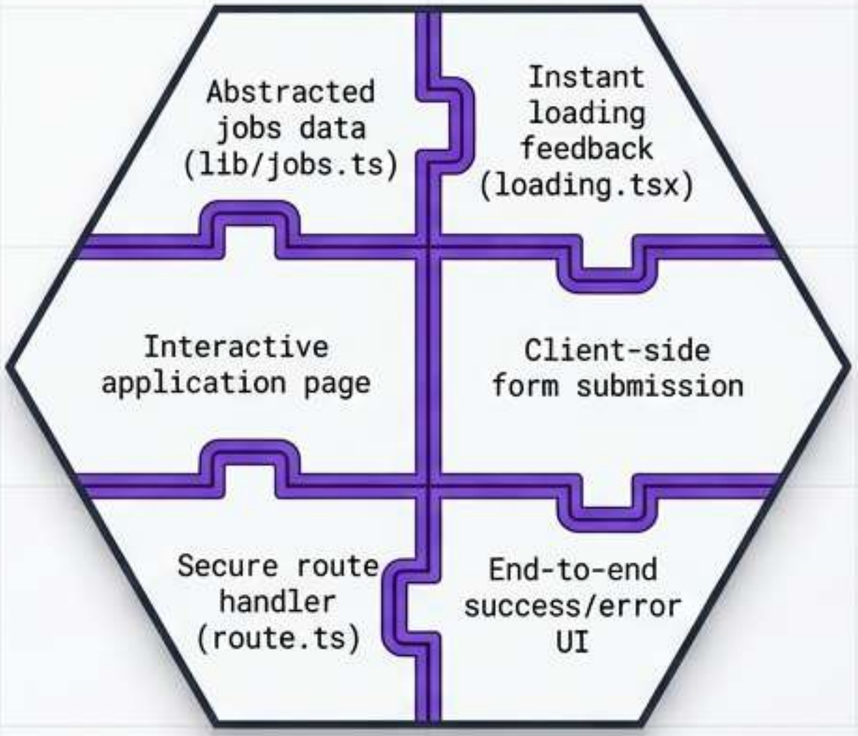


This feedback loop is the beginning of real full-stack application flow. A web app is not a collection of pages; it is a connected system.

NotebookLM

Today’s Practical Build Outcome

The ultimate win today is not just writing more code. It is understanding exactly how the separate pieces of a full-stack system finally wire together.



The ultimate win today is not just writing more code. It is understanding exactly how the separate pieces of a full-stack system finally wire together.

NotebookLM